

```
In [1]: num1 = 10_00_000
        print(num1)
1000000

In [2]: print(format(0.1, ".25f"))
0.10000000000000000055511151

In [3]: print(format(0.125, ".25f"))
0.12500000000000000000000000

In [4]: print(0.1 + 0.1 + 0.1 == 0.3)
False

In [5]: print(format(0.3, ".25f"))
0.29999999999999999888977698

In [6]: print(format(0.2, ".25f"))
0.20000000000000000111022302

In [7]: print(abs((0.1 + 0.1 + 0.1) - 0.3) < 0.001)
True

In [8]: print(0.125.as_integer_ratio())
(1, 8)

In [9]: print((10).__add__(25))
35

In [10]: a = float(10)
         print(a)
10.0

In [11]: c = (-4) ** 0.5
         print(c)
(1.2246467991473532e-16+2j)

In [12]: print(type(c))
<class 'complex'>

In [13]: print(c.real)
1.2246467991473532e-16

In [14]: print(c.imag)
2.0
```

```
In [15]: c = 10 + 2j
         print(type(c))
```

<class 'complex'>

```
In [16]: print(c)
```

(10+2j)

```
In [17]: try:
         print("2" < 2)
         except TypeError:
             pass
```

```
In [18]: print("2" == 2)
```

False

```
In [19]: a = 0
         b = 1
         c = 500
         d = 0
         e = 1
         f = 500
         print(id(a), id(b), id(c), id(d), id(e), id(f))
```

11370000 11370032 140006339516208 11370000 11370032 140006339515792

```
In [20]: print(0.1 * 3 - 0.3 < abs(1e-6))
```

True

```
In [21]: a = 257
         print(a is 257)
```

False

```
<>:2: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
<>:2: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
/tmp/ipykernel_113077/776835706.py:2: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
    print(a is 257)
```

```
In [22]: a = 10
         b = 0
         print(b != 0 and a / b)
```

False

```
In [23]: a = 1e-6
         b = 2e-6
         tol = 1e-7
         print(abs(a - b) <= tol)
```

False

```
In [24]: elapsed = 7835
         hours = elapsed // 3600
         minutes = (elapsed - (hours * 3600)) // 60
```

```
seconds = elapsed - ((hours * 3600) + (minutes * 60))
print(hours, minutes, seconds)
```

2 10 35

```
In [25]: elapsed = 7835
hours = elapsed // (60 * 60)
minutes = (elapsed % (60 * 60)) // 60
seconds = (elapsed % (60 * 60)) % 60
print(hours, minutes, seconds)
```

2 10 35
10 35

```
In [26]: a = None
if a == None:
    a = "N/A"
print(a)
```

N/A

```
In [27]: a = None
a = "N/A" if a is None else a
print(a)
```

N/A

```
In [28]: score = 700
rating = "Invalid"
if score < 0:
    rating = "Negative"
elif score < 580:
    rating = "Poor"
elif score < 670:
    rating = "Fair"
elif score < 740:
    rating = "Good"
elif score < 800:
    rating = "Very Good"
elif score < 850:
    rating = "Excellent"
print(rating)
```

Good

```
In [29]: elapsed = 1000000
magnitude = "Invalid"
if elapsed < 60:
    magnitude = "seconds"
elif elapsed < (60 * 60):
    magnitude = "minutes"
elif elapsed < (60 * 60 * 24):
    magnitude = "hours"
elif elapsed < (60 * 60 * 24 * 7):
    magnitude = "days"
elif elapsed > (60 * 60 * 24 * 7):
    magnitude = "weeks"
print(magnitude)
```

weeks

```
In [30]: t = [1, 2], 3, 4  
        l = list(t)  
        print(l)
```

[[1, 2], 3, 4]

```
In [31]: l[0][1] = 20  
        print(l)
```

[[1, 20], 3, 4]

```
In [32]: print(t)
```

((1, 20), 3, 4)

```
In [33]: for i in t:  
        print(id(i))
```

140006337746752
11370096
11370128

```
In [34]: for i in l:  
        print(id(i))
```

140006337746752
11370096
11370128

```
In [35]: for i, j in zip(l, t):  
        print(i is j)
```

True
True
True

```
In [36]: s = "ABCDEFGHIJ"  
        print(s[3:6:-1])
```

```
In [37]: print(s[3:5 + 1])
```

DEF

```
In [38]: m1 = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]  
        m2 = m1[0:2]  
        print(m1[0] is m2[1])
```

False

```
In [39]: m1[0] = "Python"  
        print(m1)
```

['Python', [0, 0, 0], [0, 0, 0]]

```
In [40]: print(m2)
```

```
[[0, 0, 0], [0, 0, 0]]
```

```
In [41]: m1[1][0] = 100
```

```
In [42]: print(m1)
['Python', [100, 0, 0], [0, 0, 0]]
```

```
In [43]: print(m2)
[[0, 0, 0], [100, 0, 0]]
```

```
In [44]: print("abcdef"[-1:-4])
```

```
In [45]: print("abcdef"[1:4:-1])
```

```
In [46]: print("racecar" == "racecar"[::-1])
True
```

```
In [47]: list1 = [0, 1, 2, 3, 4]
list1[0:3] = "abc"
print(list1)
['a', 'b', 'c', 3, 4]
```

```
In [48]: list1[0:3] = ["a", "b", "c"]
print(list1)
['a', 'b', 'c', 3, 4]
```

```
In [49]: list1[0:3] = ["a", "b"]
print(list1)
['a', 'b', 3, 4]
```

```
In [50]: del list1[2]
print(list1)
['a', 'b', 4]
```

```
In [51]: del list1[0:2]
print(list1)
[4]
```

```
In [52]: list1.extend("ABC")
print(list1)
[4, 'A', 'B', 'C']
```

```
In [53]: list1.insert(0, "Y0!")
print(list1)
['Y0!', 4, 'A', 'B', 'C']
```

```
In [54]: list1[:-3:-1] = "cb"
         print(list1)

['Y0!', 4, 'A', 'b', 'c']
```

```
In [ ]: import timeit

         l = []
         print(timeit.timeit("l.append(1)", globals = globals(), number = 100_000))

0.0017303909999100142
```

```
In [56]: print(len(l))

100000
```

```
In [57]: list2 = []
         print(timeit.timeit("list2.insert(0, 1)", globals = globals(), number = 10

1.8717184679999264
```

```
In [58]: print(len(list2))

100000
```

```
In [59]: print(1.249788 / 0.0018043)

692.6719503408524
```

```
In [60]: list2 = []
         print(timeit.timeit("list2.insert(len(list2), 1)", globals = globals(), nu

0.006136070999673393
```

```
In [61]: print(0.0057634 / 0.0018043)

3.194258161059691
```

```
In [62]: l1 = [[0, 1], 2, "3"]
         l2 = l1[:]
         for i in l1:
             print(id(i))

140006337674944
11370064
11434512
```

```
In [63]: for i in l2:
         print(id(i))

140006337674944
11370064
11434512
```

```
In [64]: l2[0][0] = "0"
         for i in l1:
             print(id(i))
```

```
140006337674944
11370064
11434512
```

```
In [65]: for i in l2:
          print(id(i))
```

```
140006337674944
11370064
11434512
```

```
In [66]: for i in l1:
          print(i)
```

```
['0', 1]
2
3
```

```
In [67]: for i in l2:
          print(i)
```

```
['0', 1]
2
3
```

```
In [68]: l2[1] = "22"
          for i in l1:
              print(id(i))
```

```
140006337674944
11370064
11434512
```

```
In [69]: for i in l2:
          print(id(i))
```

```
140006337674944
140006386671056
11434512
```

```
In [70]: for i in l1:
          print(i)
```

```
['0', 1]
2
3
```

```
In [71]: for i in l2:
          print(i)
```

```
['0', 1]
22
3
```

```
In [72]: l1 = [[0, 1], 2, "3"]
          l2 = l1[:]
          l2[0] = ["0", 1]
          for i in l1:
              print(id(i))
```

```
140006337826752
11370064
11434512
```

```
In [73]: for i in l2:
          print(id(i))
```

```
140006337826560
11370064
11434512
```

```
In [74]: for i in l1:
          print(i)
```

```
[0, 1]
2
3
```

```
In [75]: for i in l2:
          print(i)
```

```
['0', 1]
2
3
```

```
In [76]: l1 = [[0, 1], 2, "3"]
          l2 = l1[:]
          print(id(l1), id(l2))
```

```
140006337826816 140006337826496
140006337826496
```

```
In [77]: for i, j in zip(l1, l2):
          print(id(i), id(j))
```

```
140006337826432 140006337826432
11370064 11370064
11434512 11434512
```

```
In [78]: for i, j in zip(l1[0], l2[0]):
          print(id(i), id(j))
```

```
11370000 11370000
11370032 11370032
```

```
In [79]: l1 = [[0, 1], 2, "3"]
          l2 = l1.copy()
          print(id(l1), id(l2))
```

```
140006337833088 140006337830784
```

```
In [80]: for i, j in zip(l1, l2):
          print(id(i), id(j))
```

```
140006337614592 140006337614592
11370064 11370064
11434512 11434512
```

```
In [81]: for i, j in zip(l1[0], l2[0]):  
        print(id(i), id(j))
```

```
11370000 11370000  
11370032 11370032
```

```
In [ ]: import copy
```

```
l1 = [[0, 1], 2, "3"]  
l2 = copy.deepcopy(l1)  
print(id(l1), id(l2))
```

```
140006337834560 140006337778624
```

```
In [83]: for i, j in zip(l1, l2):  
        print(id(i), id(j))
```

```
140006337832896 140006337833024  
11370064 11370064  
11434512 11434512
```

```
In [84]: for i, j in zip(l1[0], l2[0]):  
        print(id(i), id(j))
```

```
11370000 11370000  
11370032 11370032
```

```
In [85]: t1 = (0, "1", "23")  
        t2 = t1[:]  
        print(t1 is t2)
```

```
True
```

```
In [86]: a, b, c, d = "abcd"  
        print(a, b, c, d)
```

```
a b c d
```

```
In [87]: s = 'FfEeDdCcBbAa'  
        lowerLetters = s[::-2]  
        upperLetters = s[-2::-2]  
        print(lowerLetters)
```

```
abcdef
```

```
In [88]: print(upperLetters)
```

```
ABCDEF
```

```
In [89]: t1 = 1, 2, 3, 4, 5, 6  
        t2 = 7, 8, 9, 10  
        t3 = 11, 12, 13, 14, 15, 16, 17  
        t = t1 + t2 + t3  
        l = list(t)  
        l[::2] = [0] * len(l[::2])  
        print(tuple(l))
```

```
(0, 2, 0, 4, 0, 6, 0, 8, 0, 10, 0, 12, 0, 14, 0, 16, 0)
```

```
In [90]: m = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
m[0][0] = 1
m[1][1] = 1
m[2][2] = 1
print(m)
```

```
[[1, 0, 0], [0, 1, 0], [0, 0, 1]]
```

```
In [91]: m = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
m1 = copy.deepcopy(m)
m1[0][0] = 1
m1[1][1] = 1
m1[2][2] = 1
print(m1)
```

```
[[1, 0, 0], [0, 1, 0], [0, 0, 1]]
```

```
In [92]: print(m)
```

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

```
In [93]: data = [(100, 'USD', 'EUR', 0.83), (100, 'USD', 'CAD', 1.27), (100, 'CAD',
USDAmountInEUR = data[0][0] * data[0][3]
print(str(data[0][0]) + " " + data[0][1] + " = " + str(USDAmountInEUR) + "
```

```
100 USD = 83.0 EUR
```

```
In [94]: USDAmountInCAD = data[1][0] * data[1][3]
print(str(data[1][0]) + " " + data[1][1] + " = " + str(USDAmountInCAD) + "
```

```
100 USD = 127.0 CAD
```

```
In [95]: CADAmountInEUR = data[2][0] * data[2][3]
print(str(data[2][0]) + " " + data[2][1] + " = " + str(CADAmountInEUR) + "
```

```
100 CAD = 65.0 EUR
```

```
In [96]: print(ord("A"))
```

```
65
```

```
In [97]: print("\N{snake}")
```

```
🐍
```

```
In [98]: print(ord("\N{snake}"))
```

```
128013
```

```
In [99]: print(hex(ord("\N{snake}")))
```

```
0x1f40d
```

```
In [100]: print("\u03b1")
```

```
α
```

```
In [101]: print("\U0001F40D")
```

```
🐍
```

```
In [102... print(int("1f40d", 16))
```

```
128013
```

```
In [103... print("\N{SNAKE} is a snake")
```

```
🐍 is a snake
```

```
In [104... print(int("7ff", 16))
```

```
2047
```

```
In [105... print(int("ffff", 16))
```

```
65535
```

```
In [106... print(int("10ffff", 16))
```

```
1114111
```

```
In [107... print(bin(2047))
```

```
0b11111111111
```

```
In [108... print(len(bin(2047)))
```

```
13
```

```
In [109... print(bin(65535))
```

```
0b1111111111111111
```

```
In [110... print(len(bin(65535)))
```

```
18
```

```
In [111... print(bin(1114111))
```

```
0b10000111111111111111
```

```
In [112... print(len(bin(1114111)))
```

```
23
```

```
In [113... print(bin(128013))
```

```
0b11111010000001101
```

```
In [114... print("\n{snaKE}")
```

```
{snaKE}
```

```
In [115... a = True  
b = True  
print(a is b)
```

```
True
```

```
In [116... a = False  
b = False
```

```
print(a is b)
```

True

```
In [117... a = True
print(id(a))
```

11120192

```
In [118... a = False
print(id(a))
```

11045056

```
In [119... b = False
print(id(b))
```

11045056

```
In [120... ?str.find
```

Docstring:

Return the lowest index in S where substring sub is found, such that sub is contained within S[start:end].

Optional arguments start and end are interpreted as in slice notation.
Return -1 on failure.

Type: method_descriptor

```
In [121... print(f"1 + 1 = {1 + 1}")
```

1 + 1 = 2

```
In [122... bid = 1.5760
ask = 1.5763
print("bid: {}, ask: {}, spread: {}".format(bid, ask, ask - bid))
```

bid: 1.576, ask: 1.5763, spread: 0.000299999999999996696

```
In [123... print("bid: {:.4f}, ask: {:.4f}, spread: {:.4f}".format(bid, ask, ask - bi
```

bid: 1.5760, ask: 1.5763, spread: 0.0003

```
In [124... print(f"bid: {bid:.4f}, ask: {ask:.4f}, spread: {ask - bid:.4f}")
```

bid: 1.5760, ask: 1.5763, spread: 0.0003

```
In [125... print(f"bid: {bid:.4f}, ask: {ask:.4f}, spread: {(ask - bid):.4f}")
```

bid: 1.5760, ask: 1.5763, spread: 0.0003

```
In [126... string1 = "ß, ß, #"
string1 = 'Π, ú, θ, ω, ν'
chars = string1.split(", ")
print(chars)
```

['Π', 'ú', 'θ', 'ω', 'ν']

```
In [127... list1 = []
for c in chars:
    list1.append(hex(ord(c)))
print(list1)

['0x3a0', '0x3cd', '0x3b8', '0x3c9', '0x3bd']
```

```
In [128... list1 = []
for c in chars:
    list1.append(c.upper())
print(list1)

['Π', 'Υ', 'Θ', 'Ω', 'Ν']
```

```
In [129... list1 = []
for c in chars:
    list1.append(c.lower())
print(list1)

['π', 'ύ', 'θ', 'ω', 'ν']
```

```
In [130... a = 42
print(f"The number {a} is not even") if a % 2 else print(f"The number {a}

The number 42 is even
```

```
In [131... a = 41
print(f"The number {a} is not even") if a % 2 else print(f"The number {a}

The number 41 is not even
```

```
In [132... a = 42
print("The number {} is not even".format(a)) if a % 2 else print("The numb

The number 42 is even
```

```
In [133... a = 41
print("The number {} is not even".format(a)) if a % 2 else print("The numb

The number 41 is not even
```

```
In [134... a = 0.3
b = 0.1
print(f"{a} / {b} = {a / b:.4f}")

0.3 / 0.1 = 3.0000
```

```
In [135... a = 42
qualifier = "is not" if a % 2 else "is"
print(f"The number {a} {qualifier} even")

The number 42 is even
```

```
In [136... a = 421
qualifier = "is not" if a % 2 else "is"
print(f"The number {a} {qualifier} even")

The number 421 is not even
```

```
In [137... print(range(10))
```

```
range(0, 10)
```

```
In [138... print(type(range(10)))
```

```
<class 'range'>
```

```
In [139... for i in range(3):  
            print(i)  
        print(i)
```

```
0
```

```
1
```

```
2
```

```
2
```

```
In [140... for i in "abc":  
            pass  
        print(i)
```

```
c
```

```
In [141... raggedArray = [[1, 2], [3, 4, 5, 6], [7, 8, 9], [10]]  
for r in range(len(raggedArray)):  
    for c in range(len(raggedArray[r])):  
        print(f"[{r}, {c}] = {raggedArray[r][c]}")  
    print("-" * 5)
```

```
[0, 0] = 1
```

```
[0, 1] = 2
```

```
-----
```

```
[1, 0] = 3
```

```
[1, 1] = 4
```

```
[1, 2] = 5
```

```
[1, 3] = 6
```

```
-----
```

```
[2, 0] = 7
```

```
[2, 1] = 8
```

```
[2, 2] = 9
```

```
-----
```

```
[3, 0] = 10
```

```
-----
```

```
In [142... identityMatrix = []  
size = 10  
for r in range(size):  
    row = []  
    for c in range(size):  
        if r == c:  
            row.append(1)  
        else:  
            row.append(0)  
    identityMatrix.append(row)  
identityMatrix
```

```
Out[142...] [[1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
             [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
             [0, 0, 1, 0, 0, 0, 0, 0, 0, 0],
             [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
             [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
             [0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
             [0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
             [0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
             [0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
             [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]]
```

```
In [143...] raggedArray = [[1, 2], [3, 4, 5, 6], [7, 8, 9], [10]]
for i, r in enumerate(raggedArray):
    for j, e in enumerate(raggedArray[i]): # e for element, cuz it's
        print(f"[{i}, {j}] = {raggedArray[i][j]}")
    print("-" * 8)
```

```
[0, 0] = 1
[0, 1] = 2
-----
[1, 0] = 3
[1, 1] = 4
[1, 2] = 5
[1, 3] = 6
-----
[2, 0] = 7
[2, 1] = 8
[2, 2] = 9
-----
[3, 0] = 10
-----
```

```
In [144...] print(f"{0.1:.4f}")
```

```
0.1000
```

```
In [145...] print("{:.4f}".format(0.1))
```

```
0.1000
```

```
In [146...] total, count = (0,) * 2
data = [10.5, 11.2, 9.8, None, 11.5, None]
for e in data:
    if e:
        total += e
        count += 1
average = total / count
print(average)
```

```
10.75
```

```
In [147...] for i, e in enumerate(data):
    if not e:
        data[i] = average
print(data)
```

```
[10.5, 11.2, 9.8, 10.75, 11.5, 10.75]
```

```
In [148... # data = [10.5, 11.2, 9.8, None, 11.5, None]
# count = sum(1 for e in data if e)
# total = sum(e for e in data if e)
# average = total / count
# print(sum, average)
```

```
In [ ]: import statistics
```

```
data = [10.5, 11.2, 9.8, None, 11.5, None]
average = statistics.fmean(e for e in data if e)
data = [average if e is None else e for e in data]
print(data)
```

```
[10.5, 11.2, 9.8, 10.75, 11.5, 10.75]
```

```
In [ ]: import pandas
```

```
data = [10.5, 11.2, 9.8, None, 11.5, None]
df = pandas.DataFrame(data)
df.fillna(df.mean(), inplace = True)
print(df[0].tolist())
```

```
[10.5, 11.2, 9.8, 10.75, 11.5, 10.75]
```

```
In [151... print(len(str((1, 2))))
```

```
6
```

```
In [152... print(str((1, 2)))
```

```
(1, 2)
```

```
In [153... m, n = 4, 5
for i in range(1, m):
    for j in range(1, n):
        print(f"{i} x {j} = {i * j}")
    print("-" * 8)
```

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
-----
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
-----
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
-----
```

```
In [154... data = ([ '2021-01-01', 10, 20], [ '2021-01-02', 20, 18], [ '2021-01-03', -10
largest = 0
for e in data:
    distance = abs(e[2] - e[1])
    e.append(distance)
    if distance > largest:
        dateOfLargest = e[0]
        largest = distance

print(data)
print(dateOfLargest)

([ '2021-01-01', 10, 20, 10], [ '2021-01-02', 20, 18, 2], [ '2021-01-03', -10,
10, 20], [ '2021-01-04', 100, 102, 2], [ '2021-01-05', 20, 45, 25])
2021-01-05
```

```
In [155... data = [[10, 20], [20, 30], [35, 50], [45, 60]]
for i, r in enumerate(data):
    if i == 0:
        r.append("")
    else:
        if r[0] < data[i - 1][1]:
            r.append("down")
        elif r[0] > data[i - 1][1]:
            r.append("up")
        else:
            r.append("same")

print(data)

[[10, 20, ''], [20, 30, 'same'], [35, 50, 'up'], [45, 60, 'down']]
```

```
In [156... tuple1 = [0],
print(type(tuple1))
print(tuple1)
print(id(tuple1))
print(id(tuple1[0]))
tuple1[0][0] += 1
print(tuple1)
print(id(tuple1))
print(id(tuple1[0]))
```

```
<class 'tuple'>
([0],)
140006337550336
140004912908160
([1],)
140006337550336
140004912908160
```

```
In [157... print(hash(True))
a = True
print(hash(a))
b = True
print(hash(b))
a = b
print(hash(a))
```

1
1
1
1

```
In [158... print(hash(257))  
a = 257  
print(hash(a))  
b = 257  
print(hash(b))  
a = b  
print(hash(a))
```

257
257
257
257

```
In [159... print(hash(2570))  
a = 2570  
print(hash(a))  
b = 2570  
print(hash(b))  
a = b  
print(hash(a))
```

2570
2570
2570
2570

```
In [160... print(hash(25))  
a = 25  
print(hash(a))  
b = 25  
print(hash(b))  
a = b  
print(hash(a))
```

25
25
25
25

```
In [161... print(hash(2.5))  
a = 2.5  
print(hash(a))  
b = 2.5  
print(hex(hash(b)))  
a = b  
print(len(hex(hash(a))))
```

1152921504606846978
1152921504606846978
0x10000000000000002
18

```
In [162... for i in range(5):
            for j in range(4):
                pass
            print(i, j)
```

4 3

```
In [163... s1 = """
    "'And' and 'or' are the basic operations of logic. Together with 'no' (the
    of negation) they are a complete set of basic logical operations – all oth
    operations, no matter how complex, can be obtained by suitable combination
    – John von Neumann, The Computer and the Brain
    """

s2 = """
    Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod te
    labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exerc
    nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in repreh
    esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupid
    in culpa qui officia deserunt mollit anim id est laborum.
    """

charCounts = {}
for c in s1:
    if (ord(c) >= ord("A") and ord(c) <= ord("Z")) or (ord(c) >= ord("
        charCounts[c] = charCounts.get(c, 0) + 1
print(charCounts)
```

```
{'A': 1, 'n': 20, 'd': 4, 'a': 23, 'o': 31, 'r': 12, 'e': 27, 't': 21, 'h':
11, 'b': 7, 's': 9, 'i': 17, 'c': 10, 'p': 7, 'f': 4, 'l': 12, 'g': 6, 'T':
2, 'w': 2, 'y': 2, 'm': 6, 'x': 1, 'u': 3, 'J': 1, 'v': 1, 'N': 1, 'C': 1,
'B': 1}
```

```
In [164... d1 = {'a': 10, 'b': 20, 'c': 30}
d2 = {'d': 100, 'e': 200, 'f': 300}
d3 = {'f': 30, 'g': 40}
keysList, valuesList = [], []
for d in d1, d2, d3:
    for k, v in d.items():
        keysList.append(k)
        valuesList.append(v)
print(keysList)
print(valuesList)
```

```
['a', 'b', 'c', 'd', 'e', 'f', 'f', 'g']
[10, 20, 30, 100, 200, 300, 30, 40]
```

```
In [165... keysList, valuesList = [], []
keysList = list(d1.keys())
keysList.extend(list(d2.keys()))
keysList.extend(list(d3.keys()))
print(keysList)
```

```
['a', 'b', 'c', 'd', 'e', 'f', 'f', 'g']
```

```
In [166... valuesList = list(d1.values())
valuesList.extend(list(d2.values()))
valuesList.extend(list(d3.values()))
print(valuesList)
```

[10, 20, 30, 100, 200, 300, 30, 40]

```
In [167... print(list(d1.keys()) + list(d2.keys()) + list(d3.keys()))
print(list(d1.values()) + list(d2.values()) + list(d3.values()))
```

['a', 'b', 'c', 'd', 'e', 'f', 'f', 'g']
[10, 20, 30, 100, 200, 300, 30, 40]

```
In [168... grades = {'John': [90, 95, 98], 'Eric': [86, 84, 92], 'Michael': [90, 89],
exam = {'Eric': 99, 'John': 100}
# for k, v in grades.items():
#     if k in exam:
#         v.insert(0, exam[k])
#     else:
#         v.insert(0, None)
for k, v in grades.items():
    # score = exam.get(k)
    # grades[k] = [score] + grades[k]
    v.insert(0, exam.get(k))
print(grades)
```

{'John': [100, 90, 95, 98], 'Eric': [99, 86, 84, 92], 'Michael': [None, 90, 89, 85]}

```
In [169... for k in grades:
    print(k)
```

John
Eric
Michael

```
In [170... import string
```

```
In [171... l = ['AAPL', 'AAPL', 'Aapl', 'aapl', 'MSFT']
print(list(set(l)))
```

['Aapl', 'MSFT', 'aapl', 'AAPL']

```
In [172... l = [e.casefold() for e in l]
print(list(set(l)))
```

['msft', 'aapl']

```
In [173... se = {e.casefold() for e in l}
print(se)
```

{'msft', 'aapl'}

```
In [174... data = {'d1': {'a': 1, 'b': 2, 'c': 3}, 'd2': {'b': 20, 'c': 30, 'd': 40},
uniqueKeys = {k for v in (data["d1"], data["d2"], data["d3"]) for k in v}
print(uniqueKeys)
```

```
{'c', 'd', 'b', 'a', 'x'}
```

```
In [175... try:
            s1 = {[0, 1]}
            print(s1)
        except TypeError:
            pass
```

```
In [176... m = [0] * 3
            print(m)
            print(m[0] is m[1])
```

```
[0, 0, 0]
True
```

```
In [177... m = [[0] * 3 for _ in range(3)]
            print(m)
            print(m[0] is m[1])
            print(m[0][0] is m[0][1])
```

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
False
True
```

```
In [ ]: import pprint
```

```
m = [[10000 for i in range(3)] for _ in range(4)]
print(m)
```

```
[[10000, 10000, 10000], [10000, 10000, 10000], [10000, 10000, 10000], [10000, 10000, 10000]]
```

```
In [179... pprint.pprint(m, width = 1)
```

```
[[10000,
  10000,
  10000],
 [10000,
  10000,
  10000],
 [10000,
  10000,
  10000],
 [10000,
  10000,
  10000]]
```

```
In [180... print(m[0] is m[1])
            print(m[0][0] is m[0][1])
            # print(hash(m[0][0]), hash(m[0][1]))
            print(id(m[0][0]), id(m[0][1]))
```

```
False
True
140006339523696 140006339523696
```

```
In [181... m[0][0] = 1001
pprint.pprint(m)
```

```
[[1001, 10000, 10000],
 [10000, 10000, 10000],
 [10000, 10000, 10000],
 [10000, 10000, 10000]]
```

```
In [182... print(id(m[0][0]), id(m[0][1]))
print(m[0][0] is m[0][1])
```

```
140004912422544 140006339523696
False
```

```
In [183... data = [{'open': 100, 'high': 120, 'low': 90, 'close': 110}, {'open': 110,
ranges = []
for d in data:
    ranges.append(d['high'] - d['low'])
print(ranges)
```

```
[30, 50, 70, 90]
```

```
In [184... data = [{'open': 100, 'high': 120, 'low': 90, 'close': 110}, {'open': 110,
ranges = [d['high'] - d['low'] for d in data]
print(ranges)
```

```
[30, 50, 70, 90]
```

```
In [185... bogusComposites = {i for i in range(1, 101) for j in range(2, 10) if i % j
# print(bogusComposites)
# allNumbers = {i for i in range(11, 101)} | {1}
allNumbers = set(range(1, 101))
print(allNumbers - bogusComposites)
```

```
{1, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79,
83, 89, 97}
```

```
In [186... data = [{'symbol': 'ABCD', 'name': 'ABCD Company', 'ranking': 2, 'risk': 0}
filtered = [d for d in data if d["ranking"] > 5 and d["risk"] < 0.6]
print(filtered)
```

```
[{'symbol': 'CDEF', 'name': 'CDEF Company', 'ranking': 8, 'risk': 0.5}, {'s
ymbol': 'FGHI', 'name': 'FGHI Company', 'ranking': 10, 'risk': 0.1}, {'symb
ol': 'KLMN', 'name': 'KLMN Company', 'ranking': 6, 'risk': 0.4}, {'symbol':
'LMNO', 'name': 'LMNO Company', 'ranking': 8, 'risk': 0.4}, {'symbol': 'MNO
P', 'name': 'MNOP Company', 'ranking': 8, 'risk': 0.2}, {'symbol': 'OPQR',
'name': 'OPQR Company', 'ranking': 9, 'risk': 0.2}, {'symbol': 'RSTU', 'nam
e': 'RSTU Company', 'ranking': 7, 'risk': 0.3}, {'symbol': 'STUV', 'name':
'STUV Company', 'ranking': 8, 'risk': 0.1}]
```

```
In [187... for d in filtered:
    d["weight"] = float(f"{d["ranking"] * d["risk"]:.2f}")
print(filtered)
```

```
[{'symbol': 'CDEF', 'name': 'CDEF Company', 'ranking': 8, 'risk': 0.5, 'weight': 4.0}, {'symbol': 'FGHI', 'name': 'FGHI Company', 'ranking': 10, 'risk': 0.1, 'weight': 1.0}, {'symbol': 'KLMN', 'name': 'KLMN Company', 'ranking': 6, 'risk': 0.4, 'weight': 2.4}, {'symbol': 'LMNO', 'name': 'LMNO Company', 'ranking': 8, 'risk': 0.4, 'weight': 3.2}, {'symbol': 'MNOP', 'name': 'MNOP Company', 'ranking': 8, 'risk': 0.2, 'weight': 1.6}, {'symbol': 'OPQR', 'name': 'OPQR Company', 'ranking': 9, 'risk': 0.2, 'weight': 1.8}, {'symbol': 'RSTU', 'name': 'RSTU Company', 'ranking': 7, 'risk': 0.3, 'weight': 2.1}, {'symbol': 'STUV', 'name': 'STUV Company', 'ranking': 8, 'risk': 0.1, 'weight': 0.8}]
```

```
In [188... weightedStocks = [dict(symbol = d["symbol"], weight = d["weight"]) for d in weightedStocks]
```

```
[{'symbol': 'CDEF', 'weight': 4.0}, {'symbol': 'FGHI', 'weight': 1.0}, {'symbol': 'KLMN', 'weight': 2.4}, {'symbol': 'LMNO', 'weight': 3.2}, {'symbol': 'MNOP', 'weight': 1.6}, {'symbol': 'OPQR', 'weight': 1.8}, {'symbol': 'RSTU', 'weight': 2.1}, {'symbol': 'STUV', 'weight': 0.8}]
```

```
In [198... # weightedStocks = [dict(symbol = d["symbol"], weight = round(d["ranking"] * d["weight"])) for d in weightedStocks]
```

```
[{'symbol': 'BCDE', 'weight': 1.0}, {'symbol': 'CDEF', 'weight': 4.0}, {'symbol': 'FGHI', 'weight': 1.0}, {'symbol': 'HIJK', 'weight': 2.5}, {'symbol': 'KLMN', 'weight': 2.4}, {'symbol': 'LMNO', 'weight': 3.2}, {'symbol': 'MNOP', 'weight': 1.6}, {'symbol': 'OPQR', 'weight': 1.8}, {'symbol': 'RSTU', 'weight': 2.1}, {'symbol': 'STUV', 'weight': 0.8}]
```

```
In [190... m = [[0] * 2] * 3
print(m)
for r in range(len(m)):
    for c in range(len(m[r])):
        print(id(m[r][c]))
```

```
[[0, 0], [0, 0], [0, 0]]
11370000
11370000
11370000
11370000
11370000
11370000
11370000
```

```
In [191... m = [[{0}] * 2] * 3
print(m)
for r in range(len(m)):
    for c in range(len(m[r])):
        print(id(m[r][c]))
```

```
[[{0}, {0}], [{0}, {0}], [{0}, {0}]]
140006338825568
140006338825568
140006338825568
140006338825568
140006338825568
140006338825568
140006338825568
```

```
In [192... m[0][0] = 1
for r in range(len(m)):
    for c in range(len(m[r])):
        print(id(m[r][c]))
print(m)
```

```
11370032
140006338825568
11370032
140006338825568
11370032
140006338825568
[[1, {0}], [1, {0}], [1, {0}]]
```

```
In [193... m = [[{0} for _ in range(2)] for _ in range(3)]
print(m)
for r in range(len(m)):
    for c in range(len(m[r])):
        print(id(m[r][c]))

m[0][0] = 1
print(m)
```

```
[[{0}, {0}], [{0}, {0}], [{0}, {0}]]
140004912778144
140004912781056
140004912778368
140004912779488
140004912777696
140004912780608
[[1, {0}], [{0}, {0}], [{0}, {0}]]
```

```
In [194... m = [[{0} for _ in range(2)] for _ in range(3)]
print(m)
for r in range(len(m)):
    for c in range(len(m[r])):
        print(id(m[r][c]))

m[0][0] = {1}
print(m)
```

```
[[{0}, {0}], [{0}, {0}], [{0}, {0}]]
140004912778144
140004912776352
140004912779264
140004912781728
140004912779712
140004912780384
[[{1}, {0}], [{0}, {0}], [{0}, {0}]]
```

```
In [195... m = [[0 for _ in range(2)] for _ in range(3)]
for r in range(len(m)):
    for c in range(len(m[r])):
        print(id(m[r][c]))
```

11370000
11370000
11370000
11370000
11370000
11370000

```
In [196... m = [[10000 for _ in range(2)] for _ in range(3)]
            for r in range(len(m)):
                for c in range(len(m[r])):
                    print(id(m[r][c]))
            m[0][0] = 1
            print(m)
```

140004912417456
140004912417456
140004912417456
140004912417456
140004912417456
140004912417456
140004912417456
[[1, 10000], [10000, 10000], [10000, 10000]]

```
In [197... m = [{0} for _ in range(2)] for _ in range(3)]
            for r in range(len(m)):
                for c in range(len(m[r])):
                    print(id(m[r][c]))
```

140004912781056
140004912779264
140004912776352
140004912781728
140004912779712
140004912780384